

September 30, 2026

1 CS 273 Project 2.2

Users' perception analysis of cyberattack consequences using NLP.

This notebook has one human similarity matrix, 24 BOW/TF-IDF matrices, 4 WordNet matrices, and Pearson correlations between each NLP matrix and the human matrix.

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import nltk

from itertools import product
from functools import lru_cache
from scipy.stats import spearmanr, pearsonr
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from nltk import word_tokenize, pos_tag
from nltk.corpus import stopwords, wordnet as wn
from nltk.stem import PorterStemmer, WordNetLemmatizer
from IPython.display import display
```

1.1 NLTK resources

```
[2]: # These downloads only do work the first time the resources are needed.
for package in [
    "stopwords",
    "wordnet",
    "omw-1.4",
    "averaged_perceptron_tagger_eng",
    "averaged_perceptron_tagger",
]:
    nltk.download(package, quiet=True)
```

1.2 Loading the dataset

```
[3]: data = pd.read_excel("all_data.xlsx", sheet_name="transcribed data")

documents = list(data.columns[2:52])
human_vectors = data.iloc[:, 2:52].to_numpy(dtype=float)

print("Number of card-sorting rows:", human_vectors.shape[0])
print("Number of consequence descriptions:", len(documents))
pd.DataFrame({"Consequence": documents}).head()
```

Number of card-sorting rows: 226

Number of consequence descriptions: 50

C:\Python39\lib\site-packages\openpyxl\worksheet_reader.py:329: UserWarning:
Unknown extension is not supported and will be removed
warn(msg)

```
[3]:                                     Consequence
0    The cyber-attacker accessed your computer files.
1    The cyber-attacker accessed your computer prog...
2    The cyber-attacker accessed your information s...
3    The cyber-attacker caused a program on your co...
4    The cyber-attacker caused your computer progra...
```

1.3 Step 1: Human-based similarity matrix

Each consequence is represented by its provided card-sorting vector. The similarity between every pair of consequence vectors is Spearman's rank correlation, as required by the rubric.

```
[4]: def make_human_similarity(vectors):
      n = vectors.shape[1]
      matrix = np.eye(n)

      for i in range(n):
          for j in range(i + 1, n):
              similarity = spearmanr(vectors[:, i], vectors[:, j]).statistic
              if np.isnan(similarity):
                  similarity = 0.0
              matrix[i, j] = similarity
              matrix[j, i] = similarity

      return matrix

human_matrix = make_human_similarity(human_vectors)
assert human_matrix.shape == (50, 50)
assert np.allclose(human_matrix, human_matrix.T)
print("Human matrix shape:", human_matrix.shape)
```

```
pd.DataFrame(human_matrix, index=documents, columns=documents).iloc[:5, :5]
```

Human matrix shape: (50, 50)

[4]:

```
your computer files. \
The cyber-attacker accessed your computer files.
1.000000
The cyber-attacker accessed your computer progr...
0.645156
The cyber-attacker accessed your information st...
0.538703
The cyber-attacker caused a program on your com...
0.041922
The cyber-attacker caused your computer program...
0.041922
```

The cyber-attacker accessed

```
your computer programs. \
The cyber-attacker accessed your computer files.
0.645156
The cyber-attacker accessed your computer progr...
1.000000
The cyber-attacker accessed your information st...
0.325797
The cyber-attacker caused a program on your com...
0.041922
The cyber-attacker caused your computer program...
-0.029047
```

The cyber-attacker accessed

```
your information stored in an Internet site. \
The cyber-attacker accessed your computer files.
0.538703
The cyber-attacker accessed your computer progr...
0.325797
The cyber-attacker accessed your information st...
1.000000
The cyber-attacker caused a program on your com...
-0.170984
The cyber-attacker caused your computer program...
-0.170984
```

The cyber-attacker accessed

```
program on your computer to crash. \
The cyber-attacker accessed your computer files.
0.041922
```

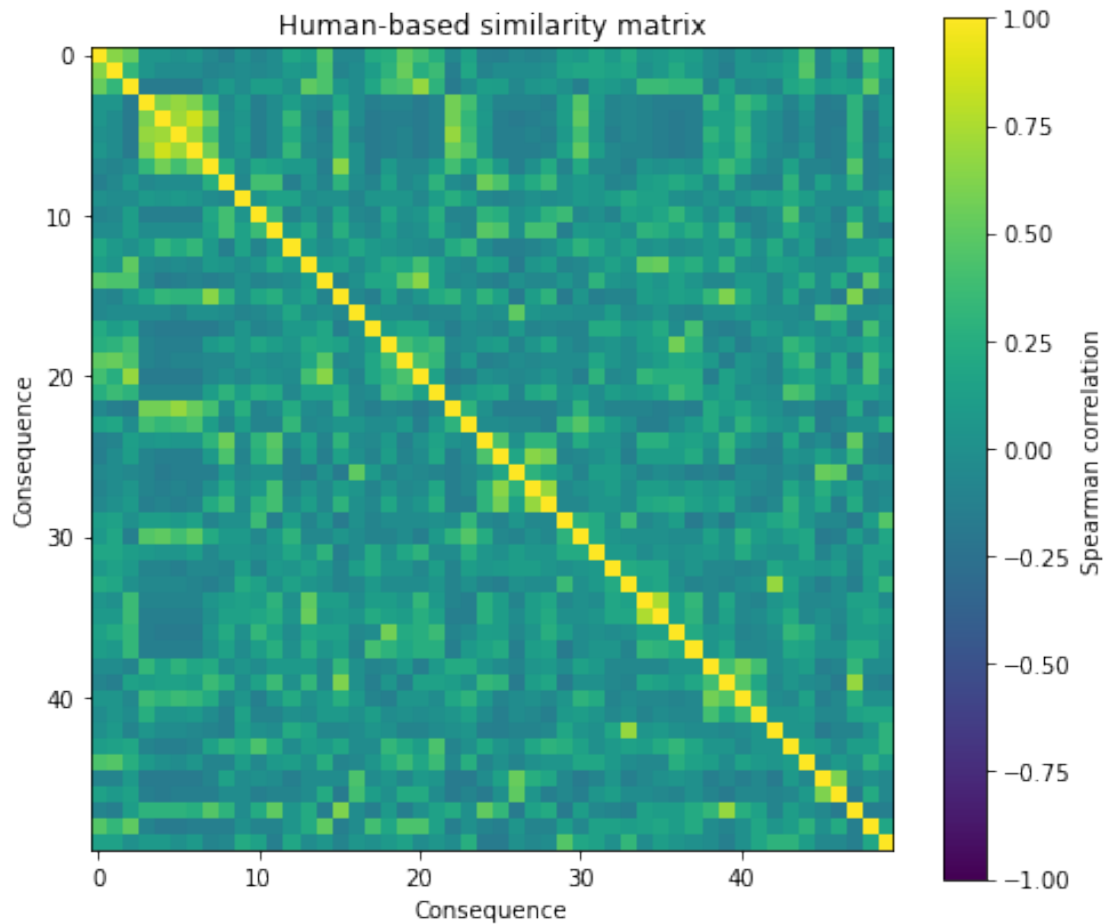
The cyber-attacker caused a

The cyber-attacker accessed your computer progr...
0.041922
The cyber-attacker accessed your information st...
-0.170984
The cyber-attacker caused a program on your com...
1.000000
The cyber-attacker caused your computer program...
0.609672

The cyber-attacker caused

your computer program to run very slowly.
The cyber-attacker accessed your computer files.
0.041922
The cyber-attacker accessed your computer progr...
-0.029047
The cyber-attacker accessed your information st...
-0.170984
The cyber-attacker caused a program on your com...
0.609672
The cyber-attacker caused your computer program...
1.000000

```
[5]: plt.figure(figsize=(7, 6))  
plt.imshow(human_matrix, vmin=-1, vmax=1)  
plt.colorbar(label="Spearman correlation")  
plt.title("Human-based similarity matrix")  
plt.xlabel("Consequence")  
plt.ylabel("Consequence")  
plt.tight_layout()  
plt.savefig("human_similarity_heatmap.png", dpi=200, bbox_inches="tight")  
plt.show()
```



1.4 Step 2: BOW and TF-IDF

For each vectorizer, the notebook uses all 12 required preprocessing configurations:

- stopwords kept or removed,
- stemming off or on,
- unigrams, bigrams, or trigrams.

Cosine similarity is used to create each pairwise similarity matrix.

```
[6]: stop_words = set(stopwords.words("english"))
      stemmer = PorterStemmer()

      def preprocess_frequency(text, remove_stopwords=False, stemming=False):
          tokens = word_tokenize(text.lower(), preserve_line=True)
          tokens = [token for token in tokens if token.isalpha()]

          if remove_stopwords:
```

```

        tokens = [token for token in tokens if token not in stop_words]
    if stemming:
        tokens = [stemmer.stem(token) for token in tokens]

    return " ".join(tokens)

```

```

[7]: frequency_matrices = {}

for remove_stopwords, stemming, ngram in product([False, True], [False, True],
↪[1, 2, 3]):
    processed = [
        preprocess_frequency(
            text,
            remove_stopwords=remove_stopwords,
            stemming=stemming,
        )
        for text in documents
    ]

    vectorizers = {
        "BOW": CountVectorizer(ngram_range=(ngram, ngram)),
        "TF-IDF": TfidfVectorizer(ngram_range=(ngram, ngram)),
    }

    for technique, vectorizer in vectorizers.items():
        vectors = vectorizer.fit_transform(processed)
        key = (technique, remove_stopwords, stemming, ngram)
        frequency_matrices[key] = cosine_similarity(vectors)

assert len(frequency_matrices) == 24
print("BOW/TF-IDF similarity matrices:", len(frequency_matrices))

```

BOW/TF-IDF similarity matrices: 24

1.5 Step 3: WordNet / GLAO

The WordNet implementation follows the required sequence:

1. `word_tokenize()` and `pos_tag()`.
2. Convert Penn Treebank tags to WordNet `n`, `v`, `a`, or `r` tags. An `s` tag is treated as `a`; unsupported tags are discarded.
3. Use the first synset returned by `wn.synsets()`.
4. Compare synsets with `path_similarity()`.
5. Greedily align every source synset with its best match and divide the score by the longer synset list.
6. Compute both directions and average them.
7. Repeat for every pair of sentences.

The four required configurations use stopword removal on/off and lemmatization on/off.

```

[8]: lemmatizer = WordNetLemmatizer()

def penn_to_wn(tag):
    # WordNet adjective satellite ('s') is treated as adjective ('a').
    if tag.lower() == "s":
        return "a"
    if tag.startswith("N"):
        return "n"
    if tag.startswith("V"):
        return "v"
    if tag.startswith("J"):
        return "a"
    if tag.startswith("R"):
        return "r"
    return None

def sentence_synsets(sentence, remove_stopwords=False, lemmatize=False):
    tokens = word_tokenize(sentence.lower(), preserve_line=True)
    tokens = [token for token in tokens if token.isalpha()]

    if remove_stopwords:
        tokens = [token for token in tokens if token not in stop_words]

    tagged_tokens = pos_tag(tokens)
    synsets = []

    for word, penn_tag in tagged_tokens:
        wn_tag = penn_to_wn(penn_tag)
        if wn_tag is None:
            continue

        if lemmatize:
            word = lemmatizer.lemmatize(word, pos=wn_tag)

        word_synsets = wn.synsets(word, pos=wn_tag)
        if word_synsets:
            synsets.append(word_synsets[0])

    return synsets

[9]: @lru_cache(maxsize=None)
def path_score(synset_a, synset_b):
    score = synset_a.path_similarity(synset_b)
    return 0.0 if score is None else float(score)

```

```

def sentence_similarity(source_synsets, target_synsets):
    if not source_synsets or not target_synsets:
        return 0.0

    total = 0.0
    for source in source_synsets:
        scores = [path_score(source, target) for target in target_synsets]
        total += max(scores) if scores else 0.0

    return total / max(len(source_synsets), len(target_synsets))

def symmetric_sentence_similarity(synsets_a, synsets_b):
    forward = sentence_similarity(synsets_a, synsets_b)
    backward = sentence_similarity(synsets_b, synsets_a)
    return (forward + backward) / 2

def make_wordnet_similarity(sentences, remove_stopwords=False, lemmatize=False):
    prepared = [
        sentence_synsets(
            sentence,
            remove_stopwords=remove_stopwords,
            lemmatize=lemmatize,
        )
        for sentence in sentences
    ]

    n = len(sentences)
    matrix = np.eye(n)

    for i in range(n):
        for j in range(i + 1, n):
            similarity = symmetric_sentence_similarity(prepared[i], prepared[j])
            matrix[i, j] = similarity
            matrix[j, i] = similarity

    return matrix

```

```

[10]: wordnet_matrices = {}

for remove_stopwords, lemmatize in product([False, True], [False, True]):
    key = ("WordNet", remove_stopwords, lemmatize)
    wordnet_matrices[key] = make_wordnet_similarity(
        documents,
        remove_stopwords=remove_stopwords,

```

```

        lemmatize=lemmatize,
    )

assert len(wordnet_matrices) == 4
print("WordNet similarity matrices:", len(wordnet_matrices))

```

WordNet similarity matrices: 4

1.6 Step 4: Compare NLP matrices with the human matrix

Only the upper triangle of each matrix is used, excluding the diagonal. Pearson correlation compares each NLP triangle with the corresponding human triangle.

```

[11]: def upper_triangle(matrix):
        indices = np.triu_indices_from(matrix, k=1)
        return matrix[indices]

def matrix_correlation(matrix_a, matrix_b):
    return pearsonr(upper_triangle(matrix_a), upper_triangle(matrix_b)).
    ↪ statistic

results = []

for (technique, remove_stopwords, stemming, ngram), matrix in
    ↪ frequency_matrices.items():
    results.append({
        "Technique": technique,
        "Stopwords Removed": remove_stopwords,
        "Normalization": "Stemming" if stemming else "None",
        "N-gram": {1: "Unigram", 2: "Bigram", 3: "Trigram"}[ngram],
        "Pearson Correlation": matrix_correlation(human_matrix, matrix),
    })

for (technique, remove_stopwords, lemmatize), matrix in wordnet_matrices.
    ↪ items():
    results.append({
        "Technique": technique,
        "Stopwords Removed": remove_stopwords,
        "Normalization": "Lemmatization" if lemmatize else "None",
        "N-gram": "-",
        "Pearson Correlation": matrix_correlation(human_matrix, matrix),
    })

results = pd.DataFrame(results)
technique_order = pd.Categorical(
    results["Technique"],

```

```

        categories=["BOW", "TF-IDF", "WordNet"],
        ordered=True,
    )
    results = results.assign(_order=technique_order).sort_values(
        ["_order", "Stopwords Removed", "Normalization", "N-gram"]
    ).drop(columns="_order").reset_index(drop=True)

    assert len(results) == 28
    print("Total NLP matrices compared:", len(results))

```

Total NLP matrices compared: 28

1.7 Correlation results

```

[12]: best_mask = results.groupby("Technique")["Pearson Correlation"].transform("max")
      best_mask = np.isclose(results["Pearson Correlation"], best_mask)

```

```

def bold_maximum(data):
    styles = pd.DataFrame("", index=data.index, columns=data.columns)
    styles.loc[best_mask, "Pearson Correlation"] = "font-weight: bold"
    return styles

```

```

display(
    results.style
        .apply(bold_maximum, axis=None)
        .format({"Pearson Correlation": "{:.3f}"})
        .hide(axis="index")
)

```

<pandas.io.formats.style.Styler at 0x1e5ebad9820>

```

[13]: best_results = results.loc[
        results.groupby("Technique")["Pearson Correlation"].idxmax()
    ].sort_values("Technique")

    print("Best configuration for each technique:")
    display(best_results.style.format({"Pearson Correlation": "{:.3f}"})
        .hide(axis="index"))

    results.to_csv("correlation_results.csv", index=False)
    print("Saved correlation_results.csv")

```

Best configuration for each technique:

<pandas.io.formats.style.Styler at 0x1e5e7f6e3d0>

Saved correlation_results.csv

1.8 Heatmaps for the best NLP configurations

```
[14]: def find_best_matrix(technique):
    technique_results = results[results["Technique"] == technique]
    best_row = technique_results.loc[technique_results["Pearson Correlation"].
    ↪idxmax()]

    remove_stopwords = bool(best_row["Stopwords Removed"])

    if technique in ["BOW", "TF-IDF"]:
        stemming = best_row["Normalization"] == "Stemming"
        ngram = {"Unigram": 1, "Bigram": 2, "Trigram": 3}[best_row["N-gram"]]
        ↪return frequency_matrices[(technique, remove_stopwords, stemming,
        ↪ngram)]

    lemmatize = best_row["Normalization"] == "Lemmatization"
    ↪return wordnet_matrices[("WordNet", remove_stopwords, lemmatize)]

for technique in ["BOW", "TF-IDF", "WordNet"]:
    matrix = find_best_matrix(technique)
    plt.figure(figsize=(6, 5))
    plt.imshow(matrix, vmin=0, vmax=1)
    plt.colorbar(label="Similarity")
    plt.title(f"Best {technique} similarity matrix")
    plt.xlabel("Consequence")
    plt.ylabel("Consequence")
    plt.tight_layout()
    filename = technique.lower().replace("-", "_") + "_best_heatmap.png"
    plt.savefig(filename, dpi=200, bbox_inches="tight")
    plt.show()
```

